



PROBLEM SESSION 4

Memory class and API in C programming.

คณะผู้จัดทำ

นัธววัฒน์	ปริมสิริคุณาวุฒิ	67070501027
วิศิษฐ์	สุวรรณเนา	67070501042
พลวิษฐ์	วัฒนเหมรัตน์	67070501067

เสนอ

ผศ. ราชวิช สโรชวิกสิต

รายงานนี้เป็นส่วนหนึ่งของรายวิชา

CPE333 ระบบปฏิบัติการ (Operating Systems)

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ภาคเรียนที่ 1 ปีการศึกษา 2569

Problem 1: Static Storage Class

Step 1: Static version (default PIE)

Step 1: Compile and run this program 3 times, copy the output and put it in your report, explain why the result show up that way.

```
#include <stdio.h>
#include <stdlib.h>
void main() {
    auto int x = 3;
    while (x > 0) {
        static int y = 5;
        y++;
        printf("The value of y is %d\n", y);
        printf("The address of y is %p\n\n", &y);
        x--;
    }
}
```

First Time

The value of y is 6
The address of y is 0x63ed7a703010

The value of y is 7
The address of y is 0x63ed7a703010

The value of y is 8
The address of y is 0x63ed7a703010

Second Time

The value of y is 6
The address of y is 0x62779c8cd010

The value of y is 7
The address of y is 0x62779c8cd010

The value of y is 8
The address of y is 0x62779c8cd010

Third Time

```
The value of y is 6
The address of y is 0x55897ec4c010

The value of y is 7
The address of y is 0x55897ec4c010

The value of y is 8
The address of y is 0x55897ec4c010
```

In the first program, `y` is declared as **static** `int y = 5`, inside the while loop. Even though the declaration is inside the loop, because it's **static**, it is initialized only once on the first time the program reaches that line, not on every iteration. The variable is stored in the **data segment** of the process, not on the **stack**, so it stands across iterations instead of being recreated each time. That's why the value of `y` accumulates across iterations (6, 7, 8) instead of resetting,

That's why the value of `y` accumulates across iterations (6, 7, 8) instead of resetting. The address of `y` stays the same across all iterations within a single run because `y` occupies one fixed location in the **data segment** for the entire lifetime of the program, since it's only initialized once, that same memory location is reused every time the loop accesses it.

However, the address of `y` changes between different runs (First/Second/Third Time) because of **ASLR** (Address Space Layout Randomization), the OS randomizes the base address of memory segments each time the program is loaded, as a security measure.

Step 2–3: Remove static and compare

Step 2: Modify the code by delete the “**static**” word, compile, and re-run the program.

Step 3: Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

```
#include <stdio.h>
#include <stdlib.h>
void main(){
    auto int x = 3;
    while (x > 0){
        int y = 5;
        y++;
        printf("The value of y is %d\n", y);
        printf("The address of y is %p\n\n", &y);
        x--;
    }
}
```

First Time

The value of y is 6
The address of y is 0x7ffca421c3a0

The value of y is 6
The address of y is 0x7ffca421c3a0

The value of y is 6
The address of y is 0x7ffca421c3a0

Second Time

The value of y is 6
The address of y is 0x7fff8e8a7f20

The value of y is 6
The address of y is 0x7fff8e8a7f20

The value of y is 6
The address of y is 0x7fff8e8a7f20

Third Time

The value of y is 6
The address of y is 0x7ffd8afa2390

The value of y is 6
The address of y is 0x7ffd8afa2390

The value of y is 6
The address of y is 0x7ffd8afa2390

We can see the value of y changed from (6, 7, 8) to (6, 6, 6) because after removing **static**, y becomes a normal **automatic** (local) variable. It gets re-declared and re-initialized to 5 at the start of each iteration, and is destroyed (goes out of scope) at the end of the block, so y++ always produces 6 every time, instead of accumulating.

The address of y stays the same across all 3 iterations because the loop body creates and destroys the same **stack** each time it runs, the **stack** pointer returns to the same offset at the start of each iteration, so the local variable ends up at the same address every time.

Discussion: Static vs non-static

Comparing the outputs

Comparing the two versions, **static** y and non-**static** y are located in different regions of the process virtual address space. The **static** variable is stored in **static** storage (typically the executable's **data segment**), while the non-**static** local variable has **automatic** storage duration and is typically placed on the **stack** in this implementation. The observed addresses therefore belong to clearly different virtual-memory regions.

In addition to the address difference, the value of y also differs between the two versions: with **static**, the value accumulates (6, 7, 8) because y retains its state between iterations; without **static**, the value resets to 6 every iteration because y is a new **automatic** variable created and destroyed each time the loop body runs.

Step 4: Compile with -no-pie

Step 4: Redo step 1–3, but this time compile the program using:

gcc -no-pie -o <your executable object name> <your c code name>

Are they different? Why or why not?

Static + -no-pie

All 3 Time for **static** y show the same output

The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

Non-static + -no-pie

Step 4: Redo step 1–3, but this time compile the program using:

gcc -no-pie -o <your executable object name> <your c code name>

Are they different? Why or why not?

Static + -no-pie

First Time

The value of y is 6
The address of y is 0x7ffcd212afc0

The value of y is 6
The address of y is 0x7ffcd212afc0

The value of y is 6
The address of y is 0x7ffcd212afc0

Second Time

The value of y is 6
The address of y is 0x7fff72501300

The value of y is 6
The address of y is 0x7fff72501300

The value of y is 6
The address of y is 0x7fff72501300

Third Time

The value of y is 6
The address of y is 0x7ffffd31a1c0

The value of y is 6
The address of y is 0x7ffffd31a1c0

The value of y is 6
The address of y is 0x7ffffd31a1c0

Static + -no-pie: the address of y is the same in all three runs (0x404018), while the default **PIE** build produced a different address between runs. This matches the lecture description of **PIE**: a **PIE** executable can be loaded at different virtual base addresses. Compiling with **-no-pie** removes that position-independent executable behavior, so the **static** variable in the executable's data area appears at a fixed virtual address in these runs.

Non-static + -no-pie: the value remains 6 on every loop iteration because y is recreated and initialized to 5 each time. Its **stack** address still differs between separate executions in the observed results. Therefore, disabling **PIE** fixes the executable image address but does not make this **automatic** local variable appear at one fixed address across runs.

Problem 2: Extern Storage Class

Step 1: Extern version (default PIE)

Step 1: Compile and run this program 3 times, copy the output and put it in your report, explain why the result show up that way.

```
#include <stdio.h>
#include <stdlib.h>

int x = 20;

void display(){
    extern int x;
    printf(" Display value of x: %d\n", x);
    printf(" Address of x (in display function): %p\n\n", &x);
}

void main(){
    extern int x;
    printf(" Print value of x: %d\n", x);
    printf(" Address of x (in main function): %p\n\n", &x);
    display();
}
```

First Time

Print value of x: 20
Address of x (in main function): 0x5cdcd979e010

Display value of x: 20
Address of x (in display function): 0x5cdcd979e010

Second Time

Print value of x: 20
Address of x (in main function): 0x5a336fbe2010

Display value of x: 20
Address of x (in display function): 0x5a336fbe2010

Third Time

```
Print value of x: 20
Address of x (in main function): 0x5fa0687b4010

Display value of x: 20
Address of x (in display function): 0x5fa0687b4010
```

The global variable `x` is defined once at file scope by `int x = 20;`. The declarations **extern** `int x;` inside `main()` and `display()` do not allocate another `x`; they declare that those functions refer to the existing **external**-definition `x`. Therefore both functions print the same value (20) and the same address within each run. With the default **PIE** build, that address changes between separate executions.

Step 2–3: Remove extern in main() and compare

Step 2: Modify the code by delete the “**extern**” word in `main()` function (line 15), compile, and re-run the program.

Step 3: Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

```
#include <stdio.h>
#include <stdlib.h>

int x = 20;
void display() {
    extern int x;
    printf(" Display value of x: %d\n", x);
    printf(" Address of x (in display function): %p\n\n", &x);
}

void main() {
    int x;
    printf(" Print value of x: %d\n", x);
    printf(" Address of x (in main function): %p\n\n", &x);
    display();
}
```

First Time

Print value of x: 0
Address of x (in main function): 0x7ffe50b2b604

Display value of x: 20
Address of x (in display function): 0x60ba8d9b5010

Second Time

Print value of x: 0
Address of x (in main function): 0x7ffc82bc36a4

Display value of x: 20
Address of x (in display function): 0x5aa10f8b2010

Third Time

Print value of x: 0
Address of x (in main function): 0x7ffe83490d24

Display value of x: 20
Address of x (in display function): 0x61ec3440d010

After **extern** is removed in `main()`, the statement becomes `int x;`, which declares a new local **automatic** variable that shadows the **global x**. This **local x** is not initialized before `printf` reads it, so its value is **indeterminate** and the read has **undefined behavior** in C; the observed value 0 must not be treated as guaranteed. `display()` still refers to the **global x** and therefore continues to print 20.

Another point we can see is the address range: `int x` in `main()` takes an address around 0x7ff... (the **stack** region), while the global `int x = 20` (seen through `display()`) takes an address around 0x5...–0x6... (the **data segment**). This confirms the two `x`'s live in completely different memory segments — local/**automatic** variables are allocated on the **stack**, while global variables are allocated in the **data segment**, consistent with the process memory layout.

Step 4: Compile with -no-pie

Step 4: Redo step 1–3, but this time compile the program using:

gcc -no-pie -o <your executable object name> <your c code name>

Are they different? Why or why not?

Extern version + -no-pie

All 3 Times

Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

Address is fixed across all runs (0x404018), value is always 20, since **-no-pie** lock the data-segment load address for the global variable, and both functions reference the same **global x**.

Non-extern in main() + -no-pie

First Time

Print value of x: 0
Address of x (in main function): 0x7ffe87061234

Display value of x: 20
Address of x (in display function): 0x404020

Second Time

Print value of x: 0
Address of x (in main function): 0x7ffe4cb0d844

Display value of x: 20
Address of x (in display function): 0x404020

Third Time

Print value of x: 0

Address of x (in main function): 0x7ffc16161854

Display value of x: 20

Address of x (in display function): 0x404020

With **-no-pie**, `display()` still accesses the **global x** and its address is fixed at 0x404020 in these runs. The **local x** inside `main()` remains an uninitialized **automatic** variable on the **stack**, so its address still changes between executions and its printed value remains **undefined behavior**. Thus **-no-pie** stabilizes the executable/global-data address observed here, but it does not turn the local **stack** variable into a fixed-address object.

Problem 3: Understand the memory allocation function.

Step 1–3: Original program (without -no-pie)

3. Understand the memory allocation function by following those steps.

Step 1: Run this program (without ‘-no-pie’):

```
#include <stdio.h>
#include <stdlib.h>
void main() {
    int *a;
    float *b;
    int c[10];
    printf("\nAddress of Pointer\n");
    printf(">>>p\n", &a);
    printf(">>>p\n", &b);
    printf("\nEffective Address\n");
    printf(">>>p\n", a);
    printf(">>>p\n", b);

    a = (int *)malloc(10 * sizeof(int));
    printf("\nAfter malloc Pointer a\n");
    printf(">>>p\n", a);
    printf(">>>p\n", &a[0]);
    printf(">>>p\n", &a[9]);
    printf("\nArray c\n");
    printf(">>>p\n", c);
    printf(">>>p\n", &c[0]);
    printf(">>>p\n", &c[9]);

    a = (int *)realloc(a, 1000 * sizeof(int));
    printf("\nAfter realloc Pointer a\n");
    printf(">>>p\n", a);
    printf(">>>p\n", &a[0]);
    printf(">>>p\n", &a[9]);
    printf(">>>p\n", &a[999]);
    free(a);
}
```

Result

Address of Pointer >>0x7ffec6167dc0 >>0x7ffec6167dc8	Step 2: Copy the output and put it in your report. Step 3: Explain, for each section (Address of Pointer, EA, array a after malloc(), and array c), what are their meanings? Why do the results show up that way? The address of the Pointer shows &a and &b, which are the addresses of the pointer variables themselves. Both variables are local variables in the same stack frame, so their addresses are close to each other in this run.
Effective Address >>0x8e00000006 >>(nil)	Effective Address prints the values stored in a and b, i.e. the addresses they would point to. At this point neither pointer has been initialized, so these printed pointer values are indeterminate; using them has undefined behavior in C. The observed 0x8e00000006 and (nil) therefore must not be interpreted as meaningful allocated addresses, and they must not be dereferenced.
After malloc Pointer a >>0x613480c8b420 >>0x613480c8b420 >>0x613480c8b444	malloc(10 * sizeof(int)) requests one contiguous heap block large enough for 10 int elements. for 0x10b33420 that show pointer a and &a[0] point to the same distance, next 0x10b33444 take distance between a[0] to a[9] for 9 blocks (don't count it self) so it's 36 bytes $0x...444 - 0x...420 = 0x24 = 36 \text{ bytes}$ that matches the result and the program allocates the continuous address.
Array c >>0x7ffec6167dd0 >>0x7ffec6167dd0 >>0x7ffec6167df4	Variable C is an array on stack so the address will stay nearly to pointer a and b, stay separately from heap like malloc. The result also shows the start point of array c and continuity of address between c[0] and c[9].
After realloc Pointer a >>0x613480c8b420 >>0x613480c8b420 >>0x613480c8b444 >>0x613480c8c3bc	In this particular run, realloc returned the same address for a, so the allocator was able to resize the allocation without moving it. This behavior is not guaranteed: realloc is allowed either to keep the same block or to allocate a new block elsewhere and copy the preserved contents. The requested new size is $4 * 1000 = 4000 \text{ bytes}$ $4000 \text{ bytes} - 4 \text{ (don't count itself)} = 3996 \text{ bytes}$

Step 4: Uncomment b allocation and run again

Step 4: Modify the code by uncomment 2 places, and re-run this program.

```
#include <stdio.h>
#include <stdlib.h>

void main() {
    int *a;
    float *b;
    int c[10];
    printf("\nAddress of Pointer\n");
    printf(">>>p\n", &a);
    printf(">>>p\n", &b);
    printf("\nEffective Address\n");
    printf(">>>p\n", a);
    printf(">>>p\n", b);

    a = (int *)malloc(10 * sizeof(int));

    printf("\nAfter malloc Pointer a\n");
    printf(">>>p\n", a);
    printf(">>>p\n", &a[0]);
    printf(">>>p\n", &a[9]);
    printf("\nArray c\n");
    printf(">>>p\n", c);
    printf(">>>p\n", &c[0]);
    printf(">>>p\n", &c[9]);

    b = (float*) malloc(10*sizeof(float));

    printf("\nAfter malloc Pointer b\n");
    printf(">>>p\n", b);
    printf(">>>p\n", &b[0]);
    printf(">>>p\n", &b[9]);

    a = (int *)realloc(a, 1000 * sizeof(int));

    printf("\nAfter realloc Pointer a\n");
    printf(">>>p\n", a);
    printf(">>>p\n", &a[0]);
    printf(">>>p\n", &a[9]);
    printf(">>>p\n", &a[999]);
    free(a);
    free(b);
}
```

Result

Address of Pointer >>0x7fff376ee820 >>0x7fff376ee828	Same as the above.
Effective Address >>0x8e00000006 >>(nil)	Same as the above.
After malloc Pointer a >>0x5e08420 >>0x5e08420 >>0x5e08444	Same as the above.
Array c >>0x7fff376ee830 >>0x7fff376ee830 >>0x7fff376ee854	Same as the above.
After malloc Pointer b >>0x5e08450 >>0x5e08450 >>0x5e08474	In this malloc b we can see that os maintain malloc continuos to a that s and at 0x5e08444 + little data and start b at 0x5e08450 and continue for 10 blocks of float (40 bytes)
After realloc Pointer a >>0x5e08480 >>0x5e08480 >>0x5e084a4 >>0x5e0941c	Step 5: Copy the output and put it in your report. Step 6: Explain why the result shows up that way. Is the result different from the previous run? Especially for the address of array a. Why or why not? In the second run, malloc for b returns a block immediately after the original allocation in the observed address layout. When a is later resized to 1000 int elements, realloc returns a different address (0x5e08480 instead of 0x5e08420). This demonstrates an important realloc property: if the existing block cannot be enlarged in place, the allocator may move the allocation to another location and preserve the old contents. The exact placement is allocator-dependent, so the addresses themselves are evidence from this run rather than a universal layout rule. $0x5e0941c - 0x5e08480 = 0xf9c = 3996$ bytes <i>Note: The assignment requires all steps to be executed on the same machine so that the observed address behavior is compared under the same environment.</i>